

Trabajo Práctico N° 2 - Diseño de Sitios Web

Informe técnico Web



Integrantes:

- MÉNDEZ, Nicolás
- GONZÁLEZ, Ulises
- ESTEBAN, Yamila Laura

Informe Técnico Web: Index

A cargo de :

- Nicolás Méndez

1. Justificación de cualquier cambio respecto de lo definido en el TP1

Durante el desarrollo de esta etapa, nos dimos cuenta que pese a tener un diseño bastante firme con respecto a los mock-ups realizados, deberíamos realizar unos pequeños ajustes en cuanto a diseño y estructura para mantener una armonía visual y satisfacer de mejor forma la experiencia del usuario

- Se realizó un pequeño ajuste en la separación y los márgenes propuestos en los mockups.
- El MockUp muestra 3 imágenes en el carrusel, pero esto no es más que de forma demostrativa. La realidad de como fue pensado fue así mismo como se ve implementado. Un carrusel el cual va mostrando de forma única las noticias. Esto con el fin de no abrumar a los adultos mayores que lean estas mismas
- Se añade un título al ingresar al sitio. Esto con el fin de que sea más representativo el sitio a cada del primer vistazo. A su vez está pensado para mejorar el posicionamiento dentro de la web
- Se agrego una pequeña separación a las secciones de trámites, cartilla y turnos. Esto con el fin de que se vea mejor la transición y no se note el corte o cambio de una foto a la otra

2. Uso de componentes JS de Bootstrap

1- Carousel (Carrusel):

- **Dónde:** En el archivo `index.html` (Sección "Carrusel de Noticias").
- **Qué hace:** Alterna automáticamente imágenes promocionales e informativas (campañas de vacunación, prevención, etc.).
- **Por qué:** Permite destacar múltiples anuncios importantes en el primer pantallazo (*above the fold*) sin obligar al usuario a hacer *scroll* excesivo

2- Collapse (Menú Hamburguesa):

- **Dónde:** En la etiqueta `<header>` y `<nav>` de todas las páginas del sitio.
- **Qué hace:** Oculta y despliega los enlaces de navegación principal en dispositivos móviles.
- **Por qué:** Es el pilar del diseño *Mobile First*. Garantiza que el encabezado no se rompa en pantallas pequeñas, manteniendo las secciones de Cartilla, Turnos y Trámites accesibles mediante un solo toque.

3- Tooltips:

- **Dónde:** En el `footer` (sobre los botones de redes sociales y WhatsApp).
- **Qué hace:** Despliega un pequeño cuadro de texto descriptivo al pasar el cursor sobre los íconos vectoriales.
- **Por qué:** Mejora la accesibilidad y brinda micro-información contextual ("Visítanos en Facebook", "Contactanos por WhatsApp") manteniendo el pie de página limpio y minimalista.

3. Uso de JS para captura de eventos

- **Dónde:** En el archivo `js/main.js` (vinculado en `loginUsuario.html` y el `<header>`).
- **Qué (Eventos capturados):** Se utilizó el evento `DOMContentLoaded` para asegurar la carga del DOM antes de ejecutar scripts. Después, se implementaron *Event Listeners* para capturar el evento `click` en los botones `#login-btn` y `#logout-btn`.
- **Por qué de la elección:** Se eligió esta manera para construir un simulador de inicio de sesión del lado del cliente (Client-Side). Al capturar el `click`, evitamos el envío por defecto del `<form>`, validamos los datos ingresados, guardamos el nombre del afiliado en el `localStorage` del navegador y manipulamos el DOM para actualizar el mensaje de bienvenida y el *header* en tiempo real. Finalmente, se utilizó `setTimeout` para ejecutar una redirección automatizada.

4. Personalización de Bootstrap mediante SASS

- **Qué se personalizó:** Se sobrescribieron estilos base y se crearon variables globales (ej. `$colorHojas`, `$colorTronco`). Se diseñó desde cero la estructura `.login-container` agregando *box-shadows* difusores, con el objetivo de mantener un mejor contraste, *border-radius* amplios (20px), degradados de fondo (`linear-gradient`) y transiciones suaves (`ease 0.3s`). Además, se controló el dimensionamiento de los SVG (`object-fit`) directamente desde la hoja de estilos.
- **Dónde:** En el archivo principal `.scss` que compila hacia `styles.css`.
- **Por qué de la elección:** Trabajar con SCSS permitió centralizar la paleta de colores de la mutual, facilitando el mantenimiento y la escalabilidad del código. Modificar las propiedades de los inputs y botones mediante SCSS (como los efectos `:hover` y `:focus`) otorgó a la interfaz un acabado "facherito", moderno e institucional, manteniendo el HTML limpio de estilos en línea.

Informe Técnico Web: Cartilla

A cargo de :

- Nicolás Méndez

1. Justificación de cualquier cambio respecto de lo definido en el TP1

Al momento de desarrollar este apartado, se buscó darle dinamismo y funcionalidad a la barra de búsqueda. Por cuestiones de tiempo y complejidad no se pudo.

Se optó por una forma estática simulando dicha funcionalidad

2. Uso de componentes JS de Bootstrap

1- Accordion (Acordeón):

- **Dónde:** En la lista de especialidades médicas (ej. Cardiología, Pediatría, Odontología).
- **Qué hace:** Agrupa a los profesionales dentro de categorías que se pueden expandir y colapsar al hacer clic. Al abrir una especialidad, el componente cierra automáticamente las demás.
- **Por qué:** Es la solución ideal para directorios extensos. Permite mantener la interfaz limpia y ordenada, mostrando únicamente la información que el afiliado solicita en ese momento exacto.

2- Modal (Ventana Emergente):

- **Dónde:** En los botones de "Ver detalle" o "Solicitar turno" de cada médico/institución.
- **Qué hace:** Abre una caja de diálogo superpuesta sobre la página actual, oscureciendo el fondo, para mostrar información ampliada (días de atención, dirección, mapa).
- **Por qué:** Mejora drásticamente la fluidez de navegación. Evita tener que crear una página HTML individual para cada uno de los médicos, manteniendo al usuario siempre dentro de la cartilla principal sin perder su hilo de búsqueda.

3. Uso de JS para captura de eventos

- **Dónde:** En el archivo `js/main.js` (vinculado en `loginUsuario.html` y el `<header>`).
- **Qué (Eventos capturados):** Se utilizó la captura del evento `keyup` (al soltar una tecla) en una barra de búsqueda de profesionales, o bien la captura del evento `click` sobre las tarjetas de los médicos.

- **Por qué de la elección:** En el caso de un buscador, capturar el evento `keyup` permite leer el texto ingresado en tiempo real y filtrar dinámicamente el contenido del DOM (ocultando a los médicos que no coinciden con la búsqueda) sin necesidad de recargar la página. En el caso del evento `click`, sirve para interactuar con los Modales y pasar dinámicamente el nombre del médico seleccionado hacia el formulario de reserva de turnos, logrando una experiencia de usuario interactiva y sin fricciones.

4. Personalización de Bootstrap mediante SASS

- **Qué se personalizó:** Se intervinieron los estilos predeterminados de las `.card` (tarjetas de presentación de los médicos) y de los botones del `.accordion`. Se eliminaron los bordes duros estándar de Bootstrap, reemplazándolos por sombras tenues (`box-shadow`) y esquinas redondeadas (`border-radius`). Asimismo, se modificó el comportamiento del componente Acordeón para que, al estar activo/abierto, adopte el color principal de la mutual (`$colorHojas`) en lugar del azul genérico del *framework*.
- **Dónde:** En los bloques de código SASS correspondientes a la grilla de la cartilla dentro del archivo `.scss` principal.
- **Por qué de la elección:** La cartilla médica es el núcleo funcional de la mutual. Se utilizó SASS para garantizar que los elementos de navegación transmitan la misma identidad visual, calma y profesionalidad que el resto de las pantallas institucionales, asegurando coherencia gráfica total (UI) mediante el uso de variables.

Informe Técnico Web: Turnos

A cargo de : Esteban, Yamila Laura

1. Justificación de cambios respecto del TP1.

Durante el desarrollo de la sección de turnos se realizaron algunos cambios con respecto al diseño presentado en el Trabajo Práctico 1 (TP1). Estos cambios surgieron para mejorar el funcionamiento del sitio y facilitar las pruebas.

- **Automatización de datos del paciente**

Se incorporó una función que genera automáticamente un nombre y un DNI ficticios al confirmar un turno. Esta decisión permite simular el funcionamiento de un sistema donde el usuario ya inició sesión, evitando tener que completar esos datos manualmente en cada prueba. Además, facilita las pruebas del formulario sin afectar el funcionamiento del resto del sistema.

- **Límite del historial de turnos**

En la idea inicial el historial de turnos no tenía límite. Sin embargo, como los datos se almacenan en localStorage, se decidió conservar únicamente los últimos cinco turnos realizados. Para ello se utiliza el método `.slice(0,5)` sobre el arreglo que almacena los registros.

También se eliminó el botón Eliminar de la lista de turnos, ya que el historial se actualiza automáticamente cada vez que se registra un nuevo turno.

- **Adaptación del listado para dispositivos móviles**

Se modificó la forma de mostrar el historial de turnos según el dispositivo utilizado.

En computadoras se mantiene una tabla tradicional utilizando **table-responsive**, mientras que en teléfonos celulares la tabla se reemplaza por tarjetas verticales (**card mb-3**) que facilitan la lectura y aprovechan mejor el espacio disponible en la pantalla.

2. Uso de componentes JavaScript de Bootstrap

Para mejorar la interfaz y la experiencia del usuario se incorporaron distintos componentes de Bootstrap.

- **Navbar colapsable (Collapse)**

Qué elementos se emplearon: El botón de menú desplegable (**menú hamburguesa**) con la propiedad `"data-bs-toggle="collapse"` y el contenedor de enlaces bajo la clase `.collapse`.

Dónde se ubica: En el encabezado (`<header>`) de las tres páginas desarrolladas (turnos.html, turnos-confirmacion.html y turnos-lista.html).

Por qué se eligió: En dispositivos móviles, el espacio horizontal es muy reducido para mostrar un menú completo. Este componente permite ocultar los enlaces dentro de un solo botón y desplegarlos verticalmente solo cuando el usuario lo solicita, manteniendo la pantalla despejada.

- **Menú Desplegable (Dropdowns)**

Qué elementos se emplearon: Las clases `.dropdown`, `.dropdown-toggle`, el atributo de activación `data-bs-toggle="dropdown"` y la lista de opciones `.dropdown-menu`.

Dónde se ubica: En la opción "Turnos" de la barra de navegación (tanto en la versión de escritorio como en la de celulares).

Por qué se eligió: Permitió agrupar las distintas páginas del módulo de turnos en un único lugar. Al pulsar sobre "Turnos", se despliegan de forma ordenada los accesos para "Reservar Turno" (`turnos.html`) y "Mis Turnos" (`turnos-lista.html`), evitando saturar el menú principal con enlaces sueltos.

- **Textos de Ayuda Emergentes (Tooltips)**

Qué elementos se emplearon: El atributo `data-bs-toggle="tooltip"` y el título del mensaje `data-bs-title="..."`, vinculados desde el archivo JavaScript.

Dónde se ubica: En los íconos de redes sociales (`.boton-social`) situados en el pie de página (`<footer>`) de todas las secciones. En `main.js` se incluyó el código necesario para activarlos simultáneamente al cargar la web:

Por qué se eligió: Es un recurso visual que ofrece información adicional al usuario. Al posicionar el cursor sobre los íconos de las redes, aparece un breve recuadro flotante que anticipa el destino del enlace antes de hacer clic.

- **Ventanas Emergentes (Modal)**

Qué elementos se emplearon: La estructura de bloques para modales bajo la identificación `#modal-tramite` (`.modal`, `.modal-dialog`, `.modal-content`), administrada desde JavaScript mediante la instrucción `new bootstrap.Modal(modalEl)`.

Dónde se ubica: Vinculado en el archivo `main.js` para gestionar los botones de acceso a los trámites.

Por qué se eligió: En lugar de dirigir al usuario a una nueva pestaña o utilizar alertas nativas del navegador, se optó por este cuadro de diálogo integrado. Al presionar un botón con la clase `.btn-ingresar`, el script interrumpe la recarga de la página, detecta el trámite seleccionado mediante el atributo `data-tramite`, actualiza el texto del título y del contenido de forma personalizada, y finalmente despliega la ventana con la función `.show()`.

3. Uso de JavaScript para la Captura de Eventos

La interactividad general del sitio, la simulación del inicio de sesión y la transferencia de los datos de los turnos entre las distintas páginas se resolvieron mediante la escucha de eventos en el archivo `main.js` y scripts específicos:

- **Control de Carga Inicial del Documento (DOMContentLoaded)**

Dónde se ubica: Envuelve todo el desarrollo lógico al comienzo de `main.js` y se encuentra también en las etiquetas `<script>` ubicadas al final de `turnos-confirmacion.html` y `turnos-lista.html`.

Por qué se eligió: Su presencia es esencial para asegurar que el navegador ejecute el código de JavaScript únicamente cuando la estructura HTML se haya cargado por completo. De lo contrario, el programa intentaría buscar formularios o botones que aún no existen en la pantalla, provocando fallas en la ejecución.

- **Gestión de Inicio y Cierre de Sesión (Login / Logout)**

Dónde se ubica: En `main.js`, asignado a los controles de los botones `#boton-login` y `#boton-logout`.

Por qué se eligió:

* Al presionar Login: El script recopila la información escrita en las casillas de texto. Si los campos contienen datos, guarda el nombre de usuario en la memoria del navegador mediante `localStorage.setItem("usuario", nombre)`. Simultáneamente, oculta el formulario, activa el mensaje visual de bienvenida (Bienvenido, [Nombre]), actualiza los datos del encabezado y, tras una pausa de 2 segundos administrada con un `setTimeout`, redirige al usuario a la página principal.

*Al presionar Logout: El sistema detecta el clic en el botón de salida, elimina el registro del usuario mediante `localStorage.removeItem("usuario")` y ejecuta un `location.reload()` para restablecer el sitio a su estado inicial.

- **Validación en el Envío de Formularios (submit)**

Dónde se ubica: En el archivo `main.js`, aplicado sobre los formularios que requieren control de datos obligatorios.

Por qué se eligió: Se utilizó la instrucción `e.preventDefault()` para pausar la acción de envío automática que realiza el navegador, la cual refrescaría la página perdiendo la información escrita. Luego, mediante la función `.trim()`, se remueven los espacios en blanco accidentales y se comprueba mediante condiciones lógicas que los campos requeridos estén completos antes de continuar.

- **Lectura y Renderizado de Datos desde el Almacenamiento Local**

Dónde se ubica: En los scripts internos de turnos-confirmacion.html (para mostrar los detalles de la reserva realizada) y de turnos-lista.html (para generar el listado histórico).

Por qué se eligió: Al no contar con una base de datos en un servidor, se utilizó el almacenamiento del navegador para recuperar la información grabada previamente:

En la confirmación, se extrae el turno con **JSON.parse(localStorage.getItem("ultimoTurno"))** y se asigna el texto a las etiquetas correspondientes (**#confProfesional**, **#confEspecialidad**, etc.).

En el listado de turnos, se toma el conjunto de registros guardados, se recorre cada uno de ellos mediante un ciclo y se crean dinámicamente filas (**<tr>**) para la vista de escritorio y bloques de tarjetas (**.card**) para la vista móvil utilizando comillas invertidas (**plantillas de texto**) para inyectar variables como **\${turno.fecha}** o **\${turno.profesional}**.

4. Personalización de Bootstrap mediante SASS

En concordancia con el resto del proyecto, se adoptaron las siguientes decisiones:

Paleta de Colores Institucional: Se modificaron las variables base del framework. La variable **\$primary** se adaptó para adoptar el tono verde oscuro característico de la mutual, y se ajustó la variable **\$success** para las confirmaciones visuales.

Componentes a Medida: Se definió una clase exclusiva llamada **.card-roble** que se aplicó en las estructuras de los formularios y confirmaciones. Esta regla proporciona bordes redondeados específicos, márgenes internos amplios (**padding: 2.5rem**) y sombras atenuadas. Para el comportamiento de los botones al pasar el cursor (:hover), se usaron las **funciones nativas darken() y lighten() de SASS**, las cuales calculan de forma automática la degradación del color sin necesidad de codificar valores manuales.

Por qué se eligió: Además de ser un requisito para la realización del presente trabajo, SASS permite trabajar con hojas de estilo modulares y ordenadas. En lugar de desarrollar un archivo CSS extenso con reglas repetitivas, o incurrir en el uso abusivo de la propiedad !important para forzar cambios sobre Bootstrap, se alteró la configuración desde la raíz del framework, logrando un archivo style.css limpio y eficiente.

Informe Técnico Web: Trámites

A cargo de Gonzales Ulises

1. JUSTIFICACIÓN DE CAMBIOS RESPECTO DEL TP1

- Se incorporó un modal de acceso con validación de DNI, que se activa al hacer clic en cada botón "Ingresar" de las tarjetas de trámites. Antes los botones eran solo visuales.
- Se amplió la sección de Preguntas Frecuentes (acordeón): se sumaron dos nuevas preguntas sobre Credencial Digital y reserva de turnos con anticipación, pasando de 2 a 4 ítems.
- Se agregó información de contacto real en el footer (dirección, teléfono y horarios de Mutual El Roble).
- Se completó el sitio con una sección de iconos SVG inline para cada tarjeta de trámite y botones de redes sociales con tooltips descriptivos.
- Pendientes para completar señalados en Cambios.txt:
logo de la mutual y ícono de usuario en la barra de navegación.

3. USO DE COMPONENTES JS DE BOOTSTRAP

El sitio implementa los siguientes 5 componentes JavaScript de Bootstrap 5.3.3, todos activados mediante atributos data-bs-* en el HTML, con inicialización complementaria en js/main.js:

3.1 NAVBAR COLAPSABLE (Collapse)

Archivo: tramites.html, línea 27

El botón hamburguesa lleva `data-bs-toggle="collapse"` y apunta a `#menu-principal`. En pantallas pequeñas (< 992px) la navbar se oculta y aparece al hacer clic en el ícono de menú.

Justificación: permite que la barra de navegación sea usable en celulares sin sacrificar el diseño horizontal en desktop.

3.2 TOOLTIPS

Archivo: tramites.html, líneas 59, 71, 84, 97 (tarjetas) y
líneas 268, 270, 273 (botones del footer)

Inicialización: js/main.js, líneas 4-5

```
document.querySelectorAll('[data-bs-toggle="tooltip"]')  
  .forEach(el => new bootstrap.Tooltip(el));
```

Textos:

- "Accedé a tus recetas de forma online" (Recetas Digitales)
- "Reservá y gestioná tus turnos médicos" (Turnos Online)
- "Autorizá tus prácticas médicas online" (Autorización)
- "Tu credencial siempre disponible" (Credencial Digital)
- "Visitanos en Facebook", "Seguinos en X",
"Contactanos por WhatsApp" (footer)

Justificación: mejoran la accesibilidad al dar contexto sobre la función de cada elemento sin saturar visualmente el diseño.

3.3 MODAL

Archivo: tramites.html, líneas 288-312 (`#modal-tramite`)

Activación: js/main.js, líneas 12-25

```
new bootstrap.Modal(modalEl) + ventanaModal.show()
```

El modal contiene un formulario con campos DNI/Credencial y Contraseña. Su título y texto se generan dinámicamente según el trámite seleccionado (ver punto 4).

Justificación: simula el ingreso seguro a cada módulo sin redireccionar a otra página, lo que mejora la experiencia de usuario y permite mostrar lógica JS de captura de eventos.

3.4 ACORDEÓN (Accordion / Collapse)

Archivo: tramites.html, líneas 164-213 (#faq)

4 ítems: r1 (Autorización de Estudios), r2 (Reintegros),
r3 (Credencial Digital), r4 (Turnos Online)

Controlado por data-bs-toggle="collapse" y data-bs-parent="#faq" (solo un ítem abierto a la vez).

Justificación: organiza el contenido de Preguntas Frecuentes de forma compacta, evitando que el usuario tenga que desplazarse por texto largo.

3.5 CARRUSEL (Carousel)

Archivo: tramites.html, líneas 224-243 (#carrusel-fotos)

Visible solo en mobile (clase d-md-none).

data-bs-ride="carousel" activa el avance automático.

Incluye controles prev/next para navegación manual.

Justificación: en pantallas pequeñas el carrusel es más eficiente que mostrar 3 imágenes apiladas verticalmente.

En tablet y desktop se muestra la grilla de 3 columnas.

4. USO DE JAVASCRIPT PARA CAPTURA DE EVENTOS

Archivo: js/main.js

4.1 EVENTO DOMContentLoaded

Línea 1: `document.addEventListener("DOMContentLoaded", () => { ... })`

Todo el código JS corre dentro de este callback, garantizando que el DOM esté completamente cargado antes de buscar elementos por ID o clase.

4.2 EVENTO click EN BOTONES "INGRESAR"

Líneas 17-25

```
document.querySelectorAll(".btn-ingresar")
    .forEach(boton => boton.addEventListener("click", (e) => { ... })))
```

Al hacer clic en cualquier botón "Ingresar" de la grilla:

- Se previene la acción por defecto (`e.preventDefault()`).
- Se lee el atributo `data-tramite` del botón clickeado.
- Se actualiza dinámicamente el título del modal (`tituloModal.innerHTML`) y el texto explicativo (`textoModal.innerHTML`) con el nombre del trámite.
- Se abre el modal con `ventanaModal.show()`.

Justificación: permite reutilizar un único modal para los 8 trámites distintos sin duplicar código HTML.

4.3 EVENTO click EN BOTÓN "INICIAR TRÁMITE" (validación)

Líneas 28-36

```
document.getElementById("boton-enviar")
```

```
.addEventListener("click", () => { ... })
```

Al hacer clic en "Iniciar Trámite" dentro del modal:

- Se obtiene el valor del campo DNI y se aplica `.trim()`.
- Si está vacío, se muestra un alert de error.
- Si tiene valor, se muestra un alert de confirmación con el número ingresado y se cierra el modal con `ventanaModal.hide()`.

Justificación: simula una validación real de formulario del lado del cliente antes de enviar los datos.

4.4 GUARDA DE COMPATIBILIDAD ENTRE PÁGINAS

Líneas 8-9

```
const modalEl = document.getElementById("modal-tramite");  
if (!modalEl) return;
```

Si el modal no existe en la página actual (como en `index.html`), el script se detiene aquí y no ejecuta el código del modal.

Los tooltips (punto 4.1) siguen funcionando en todas las páginas porque se inicializan antes de esta guarda.

5. PERSONALIZACIÓN DE BOOTSTRAP VÍA SASS

Archivo: `scss/styles.scss`

La personalización se realizó sobrescribiendo variables de Bootstrap ANTES del `@import`, lo que permite que Bootstrap compile con los valores del proyecto en lugar de los predeterminados.

5.1 VARIABLE \$primary → #269482 (verde institucional)

Bootstrap usa este valor en botones, links activos, bordes de foco y otros componentes que dependen del color primario.

Justificación: el verde es el color corporativo de Mutual

El Roble, por lo que reemplazar el azul predeterminado

garantiza consistencia visual sin sobrescribir clase por clase.

5.2 VARIABLE \$body-bg → #f8f6f2 (crema/marfil)

Define el fondo general del <body> en todo el sitio.

Justificación: evita el blanco puro, generando un tono más cálido que refuerza la identidad visual de la marca.

5.3 VARIABLE \$font-family-sans-serif → 'AR One Sans', sans-serif

Reemplaza la fuente base de Bootstrap (sistema) por una tipografía personalizada de Google Fonts.

Justificación: AR One Sans es legible en pantalla, moderna y coherente con el perfil institucional del sitio.

5.4 VARIABLES PROPIAS: \$color-footer y \$color-hover

\$color-footer: #794721 — marrón para el fondo del footer.

\$color-hover: #c07d46 — marrón claro para hover de botones y redes sociales.

Estas variables no son de Bootstrap; se definieron en el proyecto para poder reutilizar los colores en múltiples selectores con SCSS sin repetir el valor hexadecimal.

5.5 NESTING (anidamiento SCSS)

Las clases `.tarjeta-tramite`, `.btn-tramite` y `.boton-social` usan nesting para escribir los estados `:hover` dentro del bloque del selector padre, lo que mejora la legibilidad.

Ejemplo:

```
.tarjeta-tramite {  
  ...  
  &:hover { transform: translateY(-4px); }  
}
```

Para compilar el SCSS a CSS:

```
npm install bootstrap
```

```
npx sass scss/styles.scss css/styles.css
```


Publicación del Proyecto

URL del proyecto publicado: <https://nikerooo.github.io/>

Justificación de la elección del repositorio (GitHub Pages):

Automatización y Sincronización: Al emplear GitHub para el control de versiones de los archivos de trabajo, la activación de GitHub Pages permite un despliegue directo. Cada actualización que se sube al repositorio se refleja de manera inmediata en el sitio web publicado.

Compatibilidad con Sitios Estáticos: Dado que la plataforma de la mutual resuelve toda su lógica, navegación y persistencia de datos (como el acceso de usuarios y la agenda de turnos) en el entorno del navegador mediante JavaScript y localStorage (sin necesidad de servidores backend o bases de datos externas), GitHub Pages constituye la opción ideal. Ofrece un entorno seguro con certificado HTTPS preconfigurado y alta velocidad de respuesta de forma totalmente gratuita.